



MODUL 9 · DATABASE DENGAN PRISMA 7

Connection Pooling untuk Serverless & Production



Serverless punya connection problem unik — ini cara mengatasinya di Next.js + Vercel.

Masalah Unik Serverless — Too Many Connections

Serverless function bisa spawn ratusan instance sekaligus → ratusan koneksi DB terbuka.

● ● ● Masalah: connection exhaustion di serverless

```
// — Bagaimana serverless bekerja —————  
// 1. Request masuk ke Vercel  
// 2. Vercel spawn Lambda/Function baru (cold start)  
// 3. Setiap function MEMBUKA koneksi PostgreSQL baru  
// 4. Traffic tinggi = 1000 concurrent requests  
//    = 1000 koneksi PostgreSQL terbuka sekaligus!  
// 5. PostgreSQL default max_connections = 100  
//    → ERROR: "too many connections!"  
  
// — Mengapa di server tradisional tidak masalah? —————  
// Server tradisional: SATU proses yang persist  
// Connection pool dikelola di dalam proses tersebut  
// Max 10-20 koneksi untuk ratusan requests  
  
// — Solusi: External Connection Pooler —————  
// Connection pooler = proxy yang duduk antara app dan DB  
// Mengelola pool koneksi, reuse connection yang idle  
// App connect ke pooler (banyak), pooler connect ke DB (sedikit)  
  
// [Vercel Functions] → [PgBouncer/Neon Pooler] → [PostgreSQL]  
// 1000 connections      max 20 to DB              safe!
```

Rule of thumb: jika deploy ke Vercel/serverless, WAJIB pakai connection pooler. Tanpa ini, database akan crash saat traffic spike.

Solusi Connection Pooling di Next.js

Tiga pendekatan: Neon pooler (built-in), pg Pool, atau Prisma Accelerate.

● ● ● Opsi 1: Neon — built-in pooler

```
# Neon punya dua URL:
# 1. Direct: tanpa pooler (untuk migrate)
# 2. Pooled: lewat pooler (untuk runtime)

# .env
DATABASE_URL="postgresql://USER:PASS@ep-xxx.neon.tech/db?
sslmode=require"
# ↑ Ini sudah pooled connection di Neon!

# Untuk prisma migrate: butuh direct URL
# Tambahkan di prisma.config.ts:
# datasource: {
#   url:      env("DATABASE_URL"),    // pooled
#   directUrl: env("DIRECT_URL"),      // direct (migrate)
# }

# DIRECT_URL="postgresql://USER:PASS@ep-xxx.neon.tech/db?
sslmode=require&connection_limit=1"

# lib/db.ts — pakai @prisma/adapter-neon
import { PrismaClient } from "../generated/prisma/client"
import { PrismaNeon }    from "@prisma/adapter-neon"

const adapter = new PrismaNeon({
  connectionString: process.env.DATABASE_URL!,
})
const prisma = new PrismaClient({ adapter })
```

● ● ● Opsi 2: pg Pool untuk Node.js

```
// Untuk deployment Node.js (bukan serverless)
// pg Pool mengelola koneksi secara efisien

import { PrismaClient } from "../generated/prisma/client"
import { PrismaPg }      from "@prisma/adapter-pg"
import { Pool }          from "pg"

// Buat pool sekali (singleton)
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  // Pool config:
  max:      10,    // max 10 koneksi
  min:      2,     // min 2 idle
  idleTimeoutMillis: 30000, // idle timeout
  connectionTimeoutMillis: 2000,
})

// Pass pool ke PrismaPg
const adapter = new PrismaPg(pool)
const prisma = new PrismaClient({ adapter })

// Pool reuse koneksi yang ada
// Lebih efisien dari koneksi baru setiap kali
```

Direct URL untuk prisma migrate

Connection pooler tidak kompatibel dengan migrations. Butuh direct connection untuk migrate.

prisma.config.ts — dua URL berbeda

```
import "dotenv/config"
import { defineConfig, env } from "prisma/config"

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
    seed: "tsx prisma/seed.ts",
  },
  datasource: {
    // Runtime: pakai pooled URL (Neon/Supabase pooler)
    url: env("DATABASE_URL"),

    // Migration: pakai direct URL (langsung ke PostgreSQL)
    // prisma migrate BUTUH persistent connection
    // PgBouncer/Neon pooler tidak support ini
    directUrl: env("DIRECT_URL"),
  },
})

# .env
DATABASE_URL="postgresql://USER:PASS@pooler.neon.tech/db?sslmode=require"
DIRECT_URL="postgresql://USER:PASS@ep-direct.neon.tech/db?sslmode=require"

# Vercel environment variables:
# DATABASE_URL = pooled URL (untuk production runtime)
# DIRECT_URL    = direct URL (untuk deploy migration)
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Mengapa serverless deployment bisa menyebabkan 'too many connections' di PostgreSQL?

- A) Serverless menggunakan versi PostgreSQL yang berbeda
- B) Setiap function invocation membuka koneksi baru — ratusan concurrent requests = ratusan koneksi
- C) Serverless tidak support persistent connections

Q2 — Apa fungsi directUrl di prisma.config.ts?

- A) URL untuk development lokal
- B) URL untuk prisma migrate yang butuh direct connection (bypass pooler)
- C) URL backup jika DATABASE_URL down

Q3 — Apakah pg.Pool efektif di Vercel serverless deployment?

- A) Ya — pg.Pool adalah solusi terbaik
- B) Tidak — pool dibuat per-invocation. Gunakan hosted pooler (Neon/Supabase) untuk serverless
- C) Tergantung region deployment

Tugas Mandiri

Setup connection pooling untuk production deployment.

01 Neon Setup

Di Neon dashboard: aktifkan Connection Pooling. Copy Pooled URL (port 5432) dan Direct URL. Tambahkan keduanya ke .env.

02 `prisma.config.ts`

Update `prisma.config.ts` dengan url (pooled) dan `directUrl`. Test: `bunx prisma migrate dev` berhasil (pakai `directUrl`).

03 Vercel Deploy

Di Vercel: tambahkan `DATABASE_URL` (pooled) dan `DIRECT_URL` ke environment variables. Build command: `'prisma generate && prisma migrate deploy && next build'`.

04 Load Test

Gunakan tool seperti k6 atau Artillery untuk simulasi 50 concurrent requests ke `/api/posts`. Pantau koneksi di Neon dashboard — seharusnya max 5-10.



Production checklist: connection pooler ✓, SSL enabled ✓, `DIRECT_URL` untuk migrate ✓, `DATABASE_URL` di Vercel env ✓

Yang Sudah Kita Pelajari

- ✓ Serverless = banyak koneksi serentak → connection exhaustion tanpa pooler.
- ✓ Hosted pooler (Neon/Supabase): built-in, paling mudah untuk Next.js + Vercel.
- ✓ pg.Pool untuk Node.js traditional deployment, bukan untuk serverless.
- ✓ directUrl di prisma.config.ts: untuk prisma migrate yang butuh direct connection.
- ✓ Production: DATABASE_URL = pooled, DIRECT_URL = direct. Keduanya wajib.

→ Selanjutnya: **Modul 11 — Validasi Data dengan Zod & Forms**

